

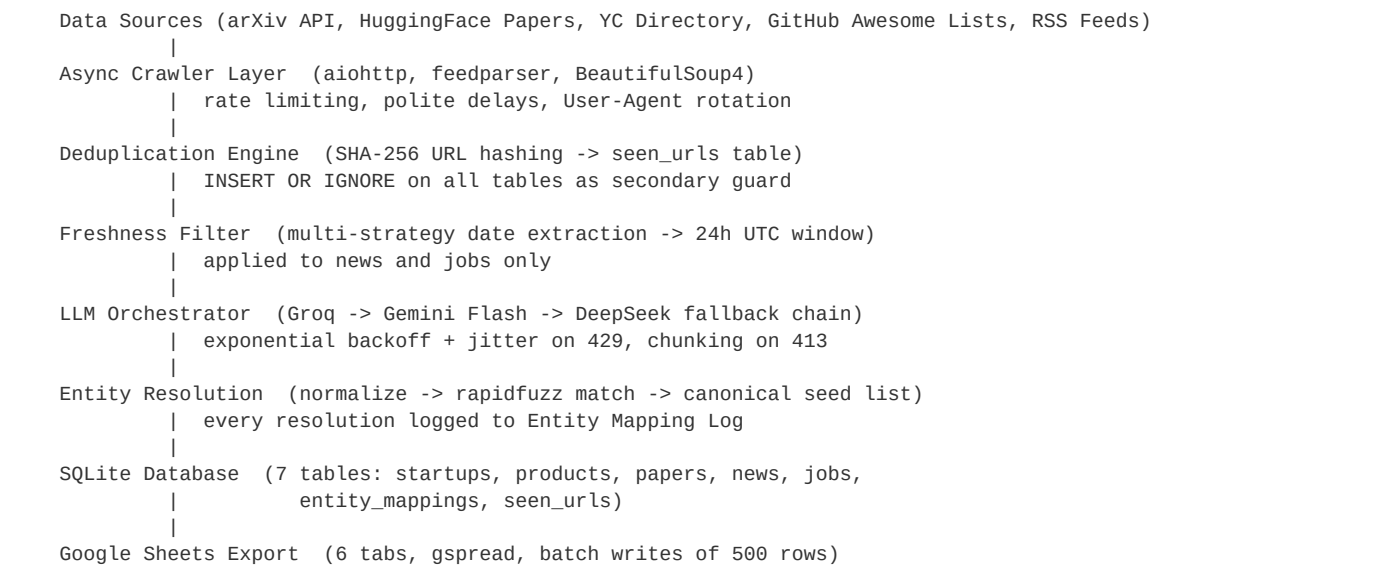
GraphOne AI Intelligence Pipeline

System Architecture Document

System Overview

This pipeline is a fully automated, modular data ingestion and intelligence extraction system designed to crawl AI ecosystem data from across the web, extract structured information using a resilient multi-provider LLM layer, deduplicate and resolve entity names to canonical forms, and export clean, traceable datasets to Google Sheets. Every record in the output is traceable back to a source URL, making hallucination detection straightforward. The system is designed as a collection of stateless, swappable components so that individual layers can be scaled or replaced without affecting the rest of the pipeline.

End to End Architecture



Data Sources and Collection Strategy

Dataset	Source	Method	Volume
Research Papers	arXiv CS.AI API + HuggingFace Papers API	arXiv returns XML via single API call. HuggingFace Papers API returns paginated JSON. Both deduplicated by paper URL.	1,100 records
Startups	Y Combinator public API	Paginated JSON API. Name, description, location, founded year per company.	1,000 records
AI Products	GitHub Awesome Lists (6 repos)	Raw markdown parsed with regex. Pricing detected from description text.	1,219 records
News	TechCrunch, VentureBeat, MIT TR, Wired, The Verge RSS	feedparser parses RSS. 24h filter applied. Full article text fetched from each article URL and extracted with BeautifulSoup.	Live, last 24h
Jobs	Jobicy (AI/ML/Data tags), HackerNews Jobs, We Work Remotely, Remoteive, AI Jobs Board	RSS feeds across 5 distinct job boards. Date extracted and compared to UTC now. Fresh only.	Live, last 24h

GitHub star counts for research papers are fetched live from the GitHub REST API at crawl time rather than copied from static pages. GitHub links are extracted from arXiv abstracts and from repository metadata returned by the HuggingFace Papers API. This ensures the metric reflects actual current repository popularity. A GitHub personal access token raises the unauthenticated rate limit from 60 to 5,000 requests per hour. Records from both paper sources are deduplicated by paper URL, so papers appearing in both arXiv and HuggingFace are stored only once.

LLM Orchestration and Error Handling

Raw text from crawled pages is passed to an LLM orchestrator that manages a three-provider fallback chain. Provider order is configurable. Groq is currently preferred as the primary provider because it offers the lowest latency on the Llama and Qwen model families. If Groq returns a rate limit error (429) or any other failure, the orchestrator switches to Gemini Flash, and then to DeepSeek if Gemini also fails. Each provider gets two attempts with exponential backoff before the next is tried. If all three providers exhaust their retries, the record is logged as failed with its source URL and skipped rather than crashing the pipeline.

Exponential backoff with full jitter means the wait time for attempt N is a random value between 0 and 2^N seconds. The jitter is critical in a distributed setting because without it, all workers that hit a rate limit simultaneously will retry at the same moment and immediately trigger the rate limit again, a pattern known as the thundering herd problem.

For pages that exceed the LLM context window (413), the pipeline runs a three-step reduction. First, BeautifulSoup strips all navigation, footer, sidebar, script, and advertisement elements and targets the main article container. Second, the cleaned text is truncated to 6,000 characters. Third, if the content is still too large, it is split into overlapping 3,000 character chunks where each chunk shares 200 characters with the next to preserve context across boundaries. Each chunk is processed independently and the resulting JSON objects are merged, with earlier chunks taking precedence for fields that appear in multiple chunks.

The LLM acts purely as an extractor and is never treated as a source of truth. Every prompt contains an explicit instruction to return null for any field not directly stated in the provided text. This is enforced at the schema level too, where all optional fields use Pydantic's Optional type with a default of None. A field like employee_count that is not mentioned on a page will always be null in the output, never a guess.

Retry and Fallback Logic

Error	Response
429 Rate limit	Exponential backoff with jitter, retry up to 2 times, then switch provider
413 Payload too large	Strip HTML noise, truncate to 6,000 chars, chunk if still too large
500 Server error	Retry with backoff, switch provider after max retries
All providers fail	Log error with source URL, skip record, continue pipeline
JSON parse failure	Strip markdown code fences, retry parse, skip if still invalid

Deduplication and Freshness

Every URL is SHA-256 hashed before any network request is made. The hash is looked up in the seen_urls table. If it exists, the URL is skipped entirely without making a request. If it does not exist, the hash is inserted and crawling proceeds. This prevents the same article or job from being processed twice even if it appears across multiple RSS feeds or across multiple pipeline runs. All main tables also have INSERT OR IGNORE with a primary key on record_id, which is itself a hash of the source URL and entity name, providing a second independent layer of deduplication at write time.

For news and jobs, the freshness filter runs a multi-strategy date extraction before applying the 24 hour window. The extractor tries each of the following strategies in order and stops at the first successful result: OpenGraph article:published_time meta tag, JSON-LD datePublished field, HTML time element with a datetime attribute, relative text patterns such as '2 hours ago' parsed with a regex, and finally dateutil brute-force parsing on text near keywords like 'published' or 'posted'. All dates are normalised to UTC before comparison against the current time.

Entity Resolution

Company names scraped from different sources appear in many inconsistent forms. The same organisation might appear as OpenAI, Open AI, OpenAI Inc., or OpenAI Incorporated across different pages. The entity resolution layer normalises raw names by converting to lowercase and stripping common legal suffixes including Inc, Ltd, Corp, LLC, and Technologies. The normalised name is then fuzzy matched against a curated seed list of 70 canonical AI company names using the rapidfuzz library's token_sort_ratio scorer, which handles word order differences gracefully. Any match scoring above 85 out of 100 is accepted and the canonical name is stored in the record. Every resolution is written to the Entity Mapping Log with the raw name, canonical name, entity type, source URL, confidence score, and timestamp. Unmatched names are stored as-is without modification.

Scaling to 500,000 Records

The MVP architecture is designed to process thousands of records within minutes on a single machine, depending on source availability and LLM provider rate limits. The architecture is designed so that scaling requires adding instances, not rewriting code. Every layer is stateless: crawlers pull URLs from a queue, LLM workers pull extracted text from a queue, and entity resolution runs as a batch job. Adding more workers at any layer increases throughput linearly.

Component	MVP (Current)	Production at 500k Scale
Database	SQLite	PostgreSQL with range partitioning by collected_at date. Separate read replicas for Sheets export.
Crawlers	Single async process	Celery distributed task queue backed by Redis. Each worker handles a subset of URLs concurrently with aiohttp.
Raw storage	Not persisted	S3-compatible object storage for raw HTML snapshots. Allows reprocessing with updated extractors without recrawling.
LLM calls	Sequential per record	Parallel worker pool with per-provider token bucket rate limiting. Dead-letter queue for records that exhaust all retries.
Deduplication	SQLite seen_urls table	Redis SET with atomic SETNX operations. Ensures exactly-once processing across all distributed workers.
Entity resolution	In-process batch	Separate microservice with its own queue. Canonical entity list stored in a shared cache accessible to all workers.
Monitoring	Print statements	Structured logging to Datadog or Grafana. Alerting on queue depth, LLM error rates, and freshness lag.

Storage Layer Design

The system uses a layered storage approach where each layer serves a specific purpose that the others cannot efficiently handle.

Layer	Technology	Purpose
Primary store	SQLite -> PostgreSQL	Structured records with full ACID compliance. Supports complex filtering queries like finding all papers with GitHub stars above a threshold published in the last 30 days.
Raw store	Local disk -> S3	Preserves original HTML at crawl time. Allows re-extraction with updated LLM prompts or improved parsers without paying the cost of recrawling.
Vector DB	Qdrant (next phase)	Enables semantic similarity search across papers, news articles, and product descriptions. Supports queries like 'find papers conceptually similar to this abstract' without exact keyword matching.
Graph DB	Neo4j (next phase)	Models relationships that are difficult to express in SQL: which startups founded after 2022 have published papers with over 500 GitHub stars, or which companies have posted jobs in the last week that match a given skill set.

Anti-Bot and Crawl Ethics

The crawler uses aiohttp with realistic User-Agent headers and polite request delays. Where official APIs or RSS feeds exist, they are preferred over HTML scraping entirely. For JavaScript-rendered sites, Playwright can replace aiohttp at the crawler layer without touching anything downstream. For sites with Cloudflare or DataDome protection, the pipeline relies on compliant acquisition methods rather than defeating CAPTCHA challenges. Residential proxy rotation can be added at the same layer swap point for IP-based rate limiting.